

Praxis: Erstes Kernelmodul entwickeln

Schreiben, bauen, laden und sicher entladen

2.03

Am Ende läuft das eigene Modul

Ein vollständiger Durchlauf mit überprüfbarem Ergebnis

01	02	03	04	05
Schreiben	Bauen	Laden	Prüfen	Entladen
hello.c	hello.ko	hello_init() läuft	Log und Status sichtbar	hello_exit() läuft

Der Testzyklus endet erst, wenn das Modul wieder sauber entladen ist.

Sichere Entwicklungsumgebung

SICHERHEIT

Testsystem

Virtuelle Maschine oder
Trainingsgerät
Snapshot oder Wiederherstellung
Keine produktiven Daten oder
Dienste

VORAUSSETZUNGEN

Werkzeuge

Compiler und make
Passende Kernel-Header oder
Build-Tree
Rechte für insmod und rmmod

VORPRÜFUNG

Umgebung prüfen

```
uname -r
```

```
ls /lib/modules/$(uname  
-r)/build
```

```
cc --version
```

Kernelversion, Header und Build-Tree müssen zueinander passen.

Projektstruktur mit zwei Dateien

TERMINAL

```
$ mkdir -p ~/kernel-training/hello  
$ cd ~/kernel-training/hello  
$ touch hello.c Makefile
```

01 hello.c

Initialisierung und Aufräumfunktion
Logmeldungen und Metadaten

02 Makefile

Modulobjekt und Build-Tree
Build- und Clean-Ziele

Quellcode und Makefile bilden das kleinste vollständige Modulprojekt.

Lebenszyklus in hello.c

QUELLCODE

```
#include <linux/init.h>
#include <linux/module.h>

static int __init hello_init(void)
{
    return 0;
}

static void __exit hello_exit(void)
{
}
```

01 hello_init()

Der Kernel ruft die Funktion beim Laden auf.
Der Rückgabewert 0 signalisiert Erfolg.

02 hello_exit()

Der Kernel ruft die Funktion beim Entladen auf.
Hier werden reservierte Ressourcen freigegeben.

Initialisierung und Aufräumen bilden den kontrollierten Lebenszyklus.

Logmeldungen für beide Übergänge

QUELLCODE

```
static int __init hello_init(void)
{
    pr_info("hello: Modul geladen\n");
    return 0;
}

static void __exit hello_exit(void)
{
    pr_info("hello: Modul entfernt\n");
}
```

01 Beim Laden

`pr_info()` schreibt die Init-Meldung in das Kernel-Log.

02 Beim Entladen

Die Exit-Meldung bestätigt den ausgeführten Aufräumpfad.

Beide Meldungen machen den Lebenszyklus später überprüfbar.

Kbuild-Makefile für hello.ko

MAKEFILE

```
obj-m += hello.o

KDIR ?= /lib/modules/$(shell uname -r)/build

all:
    $(MAKE) -C $(KDIR) M=$(CURDIR) modules

clean:
    $(MAKE) -C $(KDIR) M=$(CURDIR) clean
```

01 obj-m

Definiert das Modulziel
hello.ko.

02 KDIR

Verwendet den Build-Tree des
laufenden Kernels.

03 Tab beachten

Jede eingerückte Zeile beginnt
mit einem echten Tab.

Kbuild übersetzt das externe Modul gegen den passenden Kernel-Build.

Vollständiges Minimalmodul

HELLO.C

```
#include <linux/init.h>
#include <linux/module.h>

static int __init hello_init(void)
{
    pr_info("hello: Modul geladen\n");
    return 0;
}

static void __exit hello_exit(void)
{
    pr_info("hello: Modul entfernt\n");
}

module_init(hello_init);
module_exit(hello_exit);
MODULE_LICENSE("GPL");
```

01 Keine main()

Der Kernel steuert den Lebenszyklus.

02 Registrierung

module_init() und module_exit() verbinden die Funktionen mit dem Modul.

03 Lizenz

MODULE_LICENSE() setzt die Lizenzangabe.

Build-Ergebnis prüfen

TERMINAL

```
make
```

<code>hello.ko</code>	ladbares Kernelmodul
<code>hello.o</code>	kompiliertes Objekt
<code>hello.mod.c</code>	generierte Modulmetadaten
<code>Module.symvers</code>	Symbol- und Versionsinformationen
<code>modules.order</code>	Reihenfolge der gebauten Module

hello.ko ist die Datei, die der Kernel später lädt.

Modulinformationen vor dem Laden

TERMINAL

```
modinfo hello.ko
```

filename	Pfad zur untersuchten Moduldatei
license	im Modul hinterlegte Lizenzangabe
description	optionale Kurzbeschreibung
vermagic	Kompatibilitätsmerkmale des Builds

vermagic hilft, offensichtliche Unterschiede zum laufenden Kernel zu erkennen.

Modul laden und Initialisierung prüfen

TERMINAL

```
$ sudo insmod ./hello.ko  
$ sudo dmesg | tail -n 20
```

01 Erwartete Meldung

```
hello: Modul geladen
```

02 Was ist passiert?

Der Kernel hat hello.ko geladen und hello_init() erfolgreich ausgeführt.

Ein erfolgreicher insmod-Aufruf endet mit Rückgabewert 0 aus hello_init().

Geladenes Modul aus drei Sichten prüfen

- | | | |
|----|--|--|
| 01 | <code>lsmod grep ^hello</code> | Eintrag in der Liste geladener Module |
| 02 | <code>grep ^hello /proc/modules</code> | Kernelansicht auf die geladenen Module |
| 03 | <code>ls -la /sys/module/hello</code> | sysfs-Objekt des Moduls |

Alle drei Sichten müssen das Modul hello bestätigen.

Modul entladen und Aufräumpfad prüfen

TERMINAL

```
$ sudo rmmod hello  
$ sudo dmesg | tail -n 20  
$ lsmod | grep ^hello
```

01 Erwartete Meldung

```
hello: Modul entfernt
```

02 Abschließender Zustand

Die letzte lsmod-Abfrage liefert keine Zeile für hello.

rmmod führt `hello_exit()` aus und entfernt das Modul aus der Liste.

Typische Fehler systematisch eingrenzen

`Build-Verzeichnis fehlt`

Passende Kernel-Header oder Build-Tree installieren

`Invalid module format`

Kernelversion, Konfiguration und `vermagic` vergleichen

`Unknown symbol`

Abhängigkeiten und verwendete Kernel-API prüfen

`Operation not permitted`

Rechte sowie Signatur- und Lockdown-Regeln prüfen

`Module is in use`

Aktive Nutzer oder abhängige Module ermitteln

Fehlermeldung, Kernel-Log und Modulmetadaten führen zur Ursache.

Ergebnis sichern und Modul erweitern

ERFOLGSKRITERIEN

Abgeschlossen

hello.ko wurde erzeugt
modinfo zeigt plausible Daten
Init- und Exit-Meldung sind sichtbar
hello ist abschließend entladen

ERWEITERUNGEN

Nächste Schritte

Modulparameter ergänzen
Ungültige Werte mit -EINVAL
ablehnen
pr_warn() und pr_err() ausprobieren
Metadaten vervollständigen

AUFRÄUMEN

Arbeitsstand bereinigen

```
sudo rmmod hello  
# falls noch geladen  
  
make clean
```

Der vollständige Zyklus bildet die Grundlage für komplexere Module.